

Artificial Intelligence for (Advanced) Computer Vision

2026/27 Edition

Assignment #1

Keypoint Detectors, Feature Descriptors and Applications

1. Objective

The goal of this assignment is to introduce students to efficient keypoint detectors and local feature descriptors and to study their use in feature-based homography estimation and image stitching. The assignment also introduces modern learning-based local feature methods and requires students to compare classical and deep-learning approaches in terms of accuracy, robustness and computational efficiency.

After completing this assignment, students should be familiar with:

- Well-known handcrafted keypoint detectors and local feature descriptors.
- Binary and floating-point local descriptors and their appropriate matching strategies.
- Learned local features and learned feature matching.
- Feature matching, robust homography estimation and image warping.
- Image stitching and panorama construction.
- Experimental evaluation using standard local-feature benchmarks.
- Python/OpenCV-based computer-vision development, reproducible experiments and quantitative evaluation.

Students are also encouraged to investigate more recent state-of-the-art local feature detectors, descriptors and matchers beyond those explicitly listed below.

2. Part 1 — Keypoint Detectors and Feature Descriptors

Features in images can represent distinctive locations such as building corners, texture junctions or mountain peaks. These locations are commonly called keypoints or interest points. A keypoint detector identifies such locations, while a local feature descriptor represents the visual appearance around each detected keypoint.

A useful local feature should ideally be distinctive and sufficiently invariant to illumination, rotation, scale and moderate viewpoint changes. Local features are fundamental to applications such as image matching, panorama stitching, visual localization, SLAM and structure-from-motion.

2.1 Software Environment

All experiments must be implemented in Python. MATLAB is not required and should not be used for the submitted implementation.

The recommended environment is:

- Python 3.10 or newer.
- OpenCV 4.13.0.92 (opencv-python; opencv-contrib-python).
- NumPy.
- SciPy.
- Matplotlib.
- scikit-image, where useful for image processing and evaluation.
- pandas for storing experimental results.
- psutil or an equivalent library for optional memory measurements.
- PyTorch for SuperPoint/SuperGlue or other learned methods.

Students should provide a requirements.txt file or equivalent environment specification. A README must explain how to create the environment and execute all experiments.

Important: SIFT, ORB and KAZE are available in standard OpenCV builds. Some legacy descriptors such as BRIEF and FREAK, and some implementations of SURF, may require opencv-contrib-python and/or additional modules. Students must clearly document the exact OpenCV version and implementation used.

2.2 Baseline: SIFT and SURF

During the tutorial classes, students learned SIFT (Scale-Invariant Feature Transform), introduced by D. Lowe. SIFT detects keypoints and computes floating-point descriptors designed to be robust to scale and rotation changes.

SIFT must be used as the principal classical baseline. Students should evaluate both its detection behaviour and descriptor matching performance.

SURF (Speeded Up Robust Features) may be explored as an additional baseline. Because SURF is not part of every OpenCV distribution and may have licensing/availability limitations, it is optional. To use SURF, it may be necessary to compile the OpenCV library following the instructions in the official repository. If SURF is evaluated, students must document the implementation and the version of OpenCV used.



Figure 1: SIFT (left) and SURF (right) feature detection (extracted from [OpenCV documentation tutorial](#)).

References: D. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," IJCV, 2004; H. Bay et al., "SURF: Speeded Up Robust Features," CVIU, 2008.

2.3 FAST Keypoint Detector

FAST (Features from Accelerated Segment Test) is a high-speed corner detector designed for real-time applications. It uses a learned decision-tree strategy to rapidly reject pixels that are unlikely to be corners.

In Python/OpenCV, FAST can be accessed through the OpenCV feature-detection API. Students should investigate the effect of detector parameters, such as the intensity threshold and non-maximum suppression, and compare FAST against SIFT in terms of repeatability, number of detected points and runtime.

References: E. Rosten and T. Drummond, "Machine Learning for High-Speed Corner Detection," ECCV, 2006; E. Rosten, R. Porter and T. Drummond, "Faster and Better: A Machine Learning Approach to Corner Detection," TPAMI, 2010.

2.4 KAZE and ORB

KAZE constructs a nonlinear scale space using nonlinear diffusion and detects multiscale features. ORB (Oriented FAST and Rotated BRIEF) combines FAST-style detection with an orientation-aware binary descriptor.

Both are available in OpenCV and should be considered important alternatives to SIFT. Students should analyse the trade-off between speed, descriptor size, robustness and matching accuracy.

Reference: P. F. Alcantarilla, A. Bartoli and A. J. Davison, "KAZE Features," ECCV, 2012; E. Rublee et al., "ORB: An Efficient Alternative to SIFT or SURF," ICCV, 2011.

2.5 Binary Descriptors: BRIEF, BRISK and FREAK

Binary descriptors encode local image information through intensity comparisons. They are computationally attractive because descriptors can be matched efficiently using Hamming distance.

BRIEF is a simple binary descriptor based on comparisons between pairs of points sampled within a local patch. Students are encouraged to implement a simplified BRIEF descriptor themselves, following the sampling strategies described in the original paper. At least two sampling strategies should be investigated.

BRISK uses a handcrafted concentric-ring sampling pattern and separates sampling pairs used for orientation estimation from those used to construct the binary descriptor.

FREAK uses a retinal-inspired sampling pattern with greater sampling density near the centre and includes an orientation mechanism.

Where implementations are available, students should evaluate BRIEF, BRISK and FREAK. If a descriptor is unavailable in the student's OpenCV build, this must be stated explicitly.

The appropriate distance metric must be used: floating-point descriptors such as SIFT are normally matched with L2-type distances, whereas binary descriptors such as ORB/BRIEF/BRISK/FREAK use Hamming distance.

2.6 Deep Learning Approaches: SuperPoint and Light/SuperGlue

Students must also investigate learned local features. SuperPoint jointly detects keypoints and computes local descriptors using a convolutional neural network trained through a self-supervised homographic-adaptation strategy.

LightGlue and SuperGlue are learned feature matchers designed to establish reliable correspondences between local features extracted from two images. While SuperGlue uses a graph neural network with attention mechanisms and optimal transport to produce feature matches, LightGlue introduces a more adaptive matching strategy that dynamically adjusts its computational effort according to the difficulty of the image pair. In particular, LightGlue can use early stopping and adaptive point pruning, avoiding unnecessary computation when the matches are already sufficiently reliable. This design generally provides lower matching latency and improved computational efficiency, making LightGlue particularly attractive for real-time applications and resource-constrained systems. SuperGlue, on the other hand, provides a strong and well-established baseline for learned feature matching, but its computational cost can be less adaptive to the complexity of the input.

Thus, SuperPoint + LightGlue should be treated as a complete learned feature-extraction and matching configuration, while LightGlue itself should be evaluated as the matching component. LightGlue is a learned matcher that takes the keypoints and descriptors produced for the two images and predicts correspondences between them. The assignment should keep detector/descriptor performance conceptually separate from matcher performance when interpreting the results.

Unlike the classical OpenCV methods, these models are normally used through PyTorch-based implementations or suitable pretrained-model packages. Students do not need to train SuperPoint or LightGlue from scratch. The objective is to use a documented pretrained implementation and evaluate it fairly against the classical baselines. Some implementations and pretrained-models can be found in Hugging Face or the in the following repository links: [\[SuperPoint\]](#), [\[SuperGlue\]](#), and [\[LightGlue\]](#). Indicate which implementation was used in the report. Of note, LightGlue is pretrained for both Superpoint and SIFT descriptors. Students are encouraged to evaluate both descriptors.

Students may optionally investigate more recent learned matchers or local feature methods, provided that the baseline experiments are still completed.

2.7 Experimental Evaluation

The objective of Part 1 is not simply to display detected points. Students must quantitatively evaluate the methods.

The recommended benchmark is HPatches ([\[Download here\]](#)), which contains image sequences with controlled illumination and viewpoint changes and provides a standard setting for evaluating local features.



Figure 2: Example of a sequence extracted from the HPatches dataset. The green lines indicate the boundaries of the co-visible areas between the first image and the corresponding viewpoint-changed image.

The supplied GRAF images may also be used, particularly for qualitative analysis of geometric transformations.



For each detector/descriptor or detector-matcher combination, students should measure, where applicable:

- Number of detected keypoints.
- Detection repeatability under geometric and photometric changes.
- Number and percentage of correct matches.
- Matching precision, recall, putative matching ratio, and matching score.
- Homography estimation accuracy.
- Number of RANSAC inliers.
- Average reprojection/residual error of inliers.
- Runtime for detection, description and matching.
- Descriptor dimensionality and approximate memory usage.

Results should be reported separately for at least two transformation categories, such as illumination changes and viewpoint changes. Students should discuss why a method succeeds or fails under each condition rather than only reporting aggregate numbers.

2.8 Required Performance Metrics

To make the quantitative evaluation consistent with the established binary-feature evaluation protocol, students must report the following performance metrics for each evaluated detector/descriptor or detector/matcher configuration, whenever the required ground-truth correspondences are available. These metrics complement the assignment's existing measurements of repeatability, correct matches, homography quality and computational cost.

Putative Match Ratio (PMR). PMR measures the fraction of detected features that are initially identified as matches before geometric verification. It is defined as

$$PMR = \frac{\#N_{putative}}{\#N_{features}}.$$

Students should report the number of putative matches and the number of detected features used in the calculation. Putative correspondences should use one-to-one keypoint pairing.

Precision. Precision measures the fraction of putative matches that are geometrically correct. It is defined as

$$Precision = \frac{\#N_{correct}}{\#N_{putative}}.$$

Correctness must be determined using the available ground-truth geometry. Precision is particularly relevant to the reliability of correspondences used by robust estimation such as RANSAC.

Matching Score. Matching score measures the fraction of detected features that ultimately produce correct matches. It is defined as

$$Matching\ Score = \frac{\#N_{correct}}{\#N_{features}}.$$

It is equivalent to $PMR \times Precision$ and therefore combines the ability to generate putative matches with the ability to make those matches geometrically correct.

Recall. Recall measures how many of the possible correct correspondences are actually recovered. It is defined as

$$Recall = \frac{\#N_{correct}}{\#N_{correspondences}},$$

where $\#N_{correspondences}$ denotes the number of ground-truth correspondences available for the evaluation pair. Students should report the denominator used so that recall values are reproducible and interpretable.

Ground-truth correctness and matching protocol: For benchmark sequences with known geometric transformations, students should use the ground-truth geometry to verify putative matches. In the reference protocol, a correspondence is considered correct when the geometrically transformed point is within **3 pixels** of the matched point. If a ratio-style descriptor matching test is used, the reference protocol rejects a match when the ratio of the two best candidate distances exceeds 0.8. Students must state the actual matching threshold and correctness criterion used in their experiments.

Reporting requirement: The report must present *PMR*, *precision*, *matching score* and *recall* alongside the assignment's existing measurements. Results should be reported separately for at least two transformation categories, such as illumination changes and viewpoint changes, and should include the relevant counts or denominators so that the metrics can be reproduced. Students should discuss how detector behaviour, descriptor distinctiveness, matching criteria and geometric transformations affect these metrics.



Metric	Formula / definition	Interpretation
Putative Match Ratio	$MR = \frac{\#N_{putative}}{\#N_{features}}$	Fraction of detected features that generate putative matches.
Precision	$Precision = \frac{\#N_{correct}}{\#N_{putative}}$	Fraction of putative matches that are geometrically correct.
Matching Score	$Matching\ Score = \frac{\#N_{correct}}{\#N_{features}}$	Fraction of detected features that result in correct matches.
Recall	$Recall = \frac{\#N_{correct}}{\#N_{correspondences}}$	Fraction of available correct correspondences that are recovered.

Reference basis: J. Heinly, E. Dunn and J.-M. Frahm, "Comparative Evaluation of Binary Features," ECCV 2012, LNCS 7573, pp. 759–773. The metrics above are incorporated from the evaluation protocol described in that paper.

3. Part 2 — Feature-Based Stitching of Image Pairs

The second part of the assignment consists of developing a complete image-stitching pipeline. The system must detect distinctive features, establish correspondences, estimate the geometric transformation, warp one image with respect to another and blend the images into a panorama.

Students may use the feature detectors and descriptors evaluated in Part 1. At least three substantially different feature configurations must be compared, including one learned configuration when computationally feasible.

3.1 Image-Stitching Pipeline

Implement the following pipeline in Python:

1. Load the input images using OpenCV. Keep both colour and grayscale versions where necessary. Do not convert the final panorama to grayscale.
2. Detect keypoints in both images using the selected detector.
3. Compute local descriptors for the detected keypoints. You may use the descriptor associated with the detector or combine a detector and descriptor when supported by the method.
4. Compute descriptor correspondences using an appropriate matching strategy. For floating-point descriptors, investigate brute-force or approximate nearest-neighbour matching with an appropriate norm. For binary descriptors, use Hamming distance.
5. Apply a ratio test, cross-check or another justified filtering strategy to obtain putative correspondences. The chosen thresholds must be documented.
6. Estimate a homography using RANSAC. The homography should be estimated from point correspondences rather than from image-template matching.
7. Report the number of matches, number of RANSAC inliers, inlier ratio and average inlier reprojection error.
8. Visualize the detected keypoints and the final inlier correspondences.
9. Warp one image into the coordinate system of the other image using the estimated homography. OpenCV functions such as `cv2.warpPerspective` may be used.

10. Create an output canvas large enough to contain both images.
11. Composite the images into a panorama. A simple averaging rule in the overlap region is acceptable as a baseline; students may optionally investigate improved blending.
12. Repeat the process for multiple images to create a panorama. For three images, compare different choices of reference image.

The implementation should be written as reusable Python functions rather than as a single long script. A command-line entry point or a clearly documented main script is recommended.

3.2 Homography Estimation and RANSAC

A homography maps points between two views of a planar scene, or between views related by an appropriate projective transformation. At least four non-collinear point correspondences are required to estimate a homography.

Because local feature matching produces incorrect correspondences, a robust estimator such as RANSAC must be used. For a homography H and corresponding points p_i and p'_i , a correspondence can be considered an inlier when its reprojection error satisfies a selected threshold.

Students must report the threshold and justify it. They should also investigate how changing the RANSAC threshold affects the number of inliers and the final panorama.

OpenCV's `cv2.findHomography(..., method=cv2.RANSAC, ...)` may be used. Students should not treat the function as a black box: the report must explain the principle of RANSAC and the role of the inlier threshold.

3.3 Reference Image and Multiple-Image Stitching

When stitching three or more images, the reference image should be selected carefully. For a short sequence of three images, the central image is often a suitable reference because it can provide sufficient overlap with both neighbouring views.

For a longer sequence, homographies between adjacent views can be chained to obtain transformations relative to a common reference. Students should explain the composition of homographies and the practical consequences of accumulating geometric error.

Students should compare at least two reference-image strategies or explain, using quantitative or qualitative evidence, why their selected reference provides the most stable panorama.

3.4 Warping and Compositing

Use OpenCV's perspective-warping functionality, such as `cv2.warpPerspective`, to transform the image. Students should implement the panorama canvas and compositing logic themselves or clearly document any external implementation used.

The baseline blending method is pixel averaging in overlapping regions. Students may receive additional credit for implementing a better blending strategy, such as feathering, multiband blending or seam optimization, provided that the baseline is also evaluated.

4. Required Experimental Comparisons

The assignment should establish a fair comparison between classical and learned approaches. The minimum recommended configurations are:

- SIFT + descriptor matching.
- ORB + Hamming matching.
- FAST + a binary descriptor, such as BRIEF/BRISK/FREAK where available.
- KAZE + descriptor matching.
- SuperPoint + descriptor matching.
- SuperPoint + LightGlue.

Students may substitute SURF or other methods when justified. Any substitution must be clearly documented.

The report must distinguish between detector performance, descriptor performance and matcher performance. For example, SuperPoint and SuperGlue should not be described simply as a single 'feature detector' because SuperGlue is primarily a learned matcher.

5. Required Results and Analysis

- The report must include both quantitative and qualitative results.
- Example visualizations of keypoints for the evaluated methods.
- Examples of correct and incorrect matches.
- At least one table comparing repeatability/matching performance.
- At least one table comparing runtime and computational cost.
- Homography quality, including inlier count and reprojection error.
- Examples of final panoramas for multiple methods.
- A discussion of robustness to scale, rotation, illumination and viewpoint changes.
- A discussion of the accuracy-versus-efficiency trade-off.
- A discussion of which method is most appropriate for real-time applications and which is most appropriate when matching accuracy is the priority.

Do not compare methods only by the number of detected keypoints. A large number of keypoints is not necessarily an indication of a good local feature method.

6. Suggested Python Project Structure

A clean implementation may follow the structure below:

```
assignment1/  
  
    README.md  
  
    requirements.txt
```

```
src/  
  
    detectors.py  
  
    descriptors.py  
  
    matching.py  
  
    homography.py  
  
    stitching.py  
  
    evaluation.py  
  
    visualization.py  
  
    main.py  
  
data/  
  
results/  
  
    figures/  
  
    tables/
```

This is only a recommendation. Students may organize the project differently, but the submitted code must be modular, documented and reproducible.

7. Reproducibility Requirements

The submitted Python implementation must be reproducible on a clean environment.

- Specify the Python version.
- Provide requirements.txt or an equivalent environment file.
- State whether CPU or GPU is required.
- Provide instructions for downloading or placing the HPatches/GRAF data.
- Provide one command or clearly described sequence of commands for reproducing the main experiments.
- Record important parameter values, including detector thresholds, matching thresholds and RANSAC parameters.
- Cite any external implementation, pretrained model or code used.
- Do not include large datasets or pretrained weights in the submission unless explicitly requested by the instructor.

8. Submission Instructions

Submit by the announced due date via INFORESTUDANTE a zipped folder named LastNames_Assign#1.zip, where LastNames are the last names of the members in the group.

The submitted folder should contain:

- A Code folder containing a README with installation and execution instructions, source code with comments, requirements.txt or an equivalent environment specification, and scripts/configuration needed to reproduce the experiments.
- Input data or instructions for obtaining the required input data, subject to dataset licensing and size constraints.
- Generated output data and representative results.
- A Report containing title, authors' names, introduction, description of adopted approaches, experimental setup, results and discussion, conclusion and references.

All code must be Python. MATLAB code is not required and will not be used for assessment of the implementation.

9. Report Guidelines

a) Introduction: clearly identify the problem of keypoint detection and local descriptors, explain their importance and applications, and present the organization of the report. Suggested length: 0.75–1 page.

b) Description of Adopted Feature Detectors, Descriptors and Matchers: describe the selected methods and compare them. Explain the main contribution and underlying idea of each method, its advantages and disadvantages, and its expected computational characteristics. Avoid reproducing implementation details from the original papers. Suggested length: 3–5 pages.

c) Experimental Setup and Results: describe the datasets, evaluation protocol, parameter settings and computational environment. Present and discuss detection repeatability, matching performance, homography accuracy, computational cost and panorama quality. Suggested length: 2–5 pages.

d) Conclusion: discuss unsolved problems, limitations of the evaluated approaches, impediments to further progress and promising future directions. Suggested length: 0.75–1 page.

10. Recommended References

- D. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," International Journal of Computer Vision, 60(2), 91–110, 2004.
- H. Bay, A. Ess, T. Tuytelaars and L. Van Gool, "SURF: Speeded Up Robust Features," Computer Vision and Image Understanding, 110(3), 346–359, 2008.
- E. Rosten and T. Drummond, "Machine Learning for High-Speed Corner Detection," ECCV, 2006.
- E. Rosten, R. Porter and T. Drummond, "Faster and Better: A Machine Learning Approach to Corner Detection," IEEE TPAMI, 32, 105–119, 2010.

- P. F. Alcantarilla, A. Bartoli and A. J. Davison, “KAZE Features,” ECCV, 2012.
- E. Rublee, V. Rabaud, K. Konolige and G. Bradski, “ORB: An Efficient Alternative to SIFT or SURF,” ICCV, 2011.
- M. Calonder et al., “BRIEF: Computing a Local Binary Descriptor Very Fast,” IEEE TPAMI, 34(7), 1281–1298, 2012.
- S. Leutenegger, M. Chli and R. Siegwart, “BRISK: Binary Robust Invariant Scalable Keypoints,” ICCV, 2011.
- A. Alahi, R. Ortiz and P. Vandergheynst, “FREAK: Fast Retina Keypoint,” CVPR, 2012.
- R. Szeliski, “Computer Vision: Algorithms and Applications,” 2nd ed., Springer, 2022.
- V. Balntas, K. Lenc, A. Vedaldi and K. Mikolajczyk, “HPatches: A Benchmark and Evaluation of Handcrafted and Learned Local Descriptors,” CVPR, 2017.
- D. DeTone, T. Malisiewicz and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” CVPR Workshops, 2018.
- P. Sarlin, D. DeTone, T. Malisiewicz and A. Rabinovich, “SuperGlue: Learning Feature Matching with Graph Neural Networks,” CVPR, 2020.
- R. Szeliski and M. H. M. M. Adelson/Burt-related multiresolution blending literature may be consulted for improved panorama compositing.

11. Important Implementation Notes for Python

- Use `cv2.SIFT_create()` for SIFT.
- Use `cv2.ORB_create()` for ORB.
- Use `cv2.KAZE_create()` for KAZE.
- Use `cv2.FastFeatureDetector_create()` for FAST.
- Use `cv2.BFMatcher` or an appropriate OpenCV matcher for descriptor matching.
- Use `cv2.findHomography()` with RANSAC for robust homography estimation.
- Use `cv2.warpPerspective()` for projective image warping.
- Use `cv2.drawMatches()` or custom Matplotlib visualizations to display correspondences.
- For binary descriptors, use Hamming distance; for SIFT-like floating-point descriptors, use an L2-compatible metric.
- For SuperPoint/SuperGlue, use a documented pretrained PyTorch implementation and record the model/checkpoint used.

Students should avoid relying on `cv2.Stitcher` as the solution to Part 2 because the purpose of the assignment is to implement and understand the feature-based stitching pipeline. OpenCV’s high-level stitching API may be used only as an optional reference for qualitative comparison.

12. Assessment Focus

Assessment will emphasize understanding and experimental analysis rather than the mere production of a panorama. In particular, students should demonstrate that they understand why different detectors, descriptors and matchers behave differently and how these choices affect homography estimation and stitching.



FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE DE
COIMBRA

DEPARTAMENTO DE
ENGENHARIA ELECTROTÉCNICA
E DE COMPUTADORES

- Correctness and quality of the Python implementation.
- Quality and fairness of the experimental protocol.
- Quantitative evaluation and interpretation of results.
- Understanding of feature detection, description, matching and RANSAC.
- Quality of the final panorama and robustness of the complete pipeline.
- Reproducibility, code organization and documentation.
- Critical discussion of limitations and practical trade-offs.

GOOD LUCK AND ENJOY